B.Tech. Sem-V

Fundamentals of Operating System Design (CEUC301)

UNIT 1: CONCEPTUAL QUESTION BANK

Operating System Design, Processes and Threads

Unit / Weightage	Unit 1 | 4 hours
Question type	Conceptual, analytical and scenario-based

Syllabus coverage: Design goals (performance, scalability, reliability, security, portability and interoperability); monolithic, microkernel and hybrid architectures; PCB and TCB internals; process/thread switching overhead and performance evaluation; kernel-level and user-level threads.

A. Operating-System Design Goals

1. Explain why operating-system design requires explicit goals rather than simply implementing a collection of services.
2. How can performance and reliability conflict with one another during operating-system design? Give a realistic example.
3. Distinguish scalability from performance. Why can a system perform well for one user but scale poorly to thousands of users?
4. Explain how security requirements influence the design of operating-system interfaces and privileged operations.
5. Why is portability more than merely recompiling the operating system for another processor architecture?
6. What is interoperability in an operating-system context? Explain how it differs from portability.
7. An operating system must support old applications while introducing new hardware features. Which design goals are involved, and why might they conflict?
8. Why should design goals be evaluated in the context of a target workload instead of being treated as universally equal priorities?

B. Kernel Architectures and Design Trade-offs

9. Describe the fundamental structural idea behind a monolithic kernel. What are its principal advantages and risks?
10. Explain how a microkernel separates essential kernel mechanisms from services that can run in user space.
11. Compare monolithic and microkernel architectures in terms of communication overhead, fault isolation, maintainability and security.
12. What is meant by a hybrid kernel? Why do practical operating systems often combine ideas from more than one architecture?
13. Why does moving a service from kernel space to user space potentially improve reliability but reduce performance?
14. How can a faulty device driver affect system stability in a monolithic kernel compared with a microkernel-based design?
15. Is a microkernel automatically more secure than a monolithic kernel? Critically discuss the claim, including implementation and configuration factors.
16. An embedded system has strict response-time requirements and limited memory. Explain how these constraints affect the choice of kernel architecture.

17. A cloud server prioritizes throughput, isolation and maintainability. Identify the architectural trade-offs that should be evaluated before selecting a kernel design.
18. Explain why the labels monolithic, microkernel and hybrid do not, by themselves, provide a complete measure of an operating system's quality.

C. Process Control Block (PCB) and Thread Control Block (TCB)

19. What is a Process Control Block (PCB), and why does the operating system need one for each process?
20. Describe the major categories of information commonly maintained in a PCB and explain how each category supports process management.
21. What is a Thread Control Block (TCB)? Explain why thread-specific state must be maintained separately from process-wide state.
22. Differentiate the information typically stored in a PCB from the information stored in a TCB.
23. How do the PCB and TCB support saving and restoring execution state during a context switch?
24. Why must the operating system maintain process state, scheduling information and resource ownership consistently?
25. What problems could arise if two threads belonging to the same process were incorrectly assigned independent copies of process-wide resources?
26. Explain how PCB and TCB design can influence the overhead of process creation, thread creation and context switching.
27. Why is the PCB/TCB representation an implementation detail, even though its information is essential to operating-system functionality?

D. Process and Thread Switching; Overhead and Performance

28. Define a context switch and explain the main events that may occur during a process context switch.
29. Distinguish a mode switch from a context switch. Why should these terms not be used interchangeably?
30. Why is a context switch generally considered overhead rather than useful application work, even though it is necessary for multitasking?
31. Compare process switching and thread switching within the same process. Under what conditions can thread switching be less expensive?
32. Explain why a thread switch is not guaranteed to be cheaper than a process switch on every operating system or processor.
33. How can cache disruption, Translation Lookaside Buffer (TLB) effects and scheduler activity influence the effective cost of switching?
34. Why can increasing the frequency of preemption improve responsiveness but reduce overall throughput?
35. Which indicators would you examine to evaluate context-switching performance, and why is context-switch count alone insufficient?
36. A system shows high CPU utilization but poor application throughput. Explain how excessive switching and scheduling behaviour could contribute.
37. How do workload characteristics and hardware architecture affect whether context-switch overhead is significant?

E. Kernel-Level Threads and User-Level Threads

38. Differentiate kernel-level threads and user-level threads in terms of management, scheduling and kernel awareness.
39. Explain why user-level thread operations can be fast when thread management is performed by a user-space library.

40. What happens to other user-level threads in a process if one thread makes a blocking system call in a many-to-one threading model?
41. How can kernel-level threading improve the ability of a multithreaded process to use multiple processor cores?
42. Compare the scheduling responsibilities of a user-level thread library and the operating-system kernel.
43. Explain the principal trade-offs among many-to-one, one-to-one and many-to-many thread mapping models.
44. Why might user-level threads be suitable for applications requiring very large numbers of lightweight concurrent tasks?
45. Discuss the implications of kernel-level and user-level threads for debugging, profiling, blocking operations and application portability.

F. Integrative and Higher-Order Conceptual Questions

46. A server experiences frequent driver failures. Critically evaluate which kernel architecture characteristics could help contain failures, and identify the performance costs that may follow.
47. A team proposes moving every operating-system service into user space to maximize security. Evaluate the proposal and identify assumptions that must be tested.
48. An application has many independent tasks but frequently performs blocking I/O. Recommend a thread-management approach and justify it using concurrency, blocking and parallelism considerations.
49. A mobile operating system must balance battery life, responsiveness, security and support for diverse hardware. Explain how these goals can influence kernel and thread-management decisions.
50. Two operating systems report similar application execution times, but one has substantially more context switches. Explain why this fact alone does not establish which system is better.
51. Design a conceptual evaluation plan for comparing two kernel architectures. Identify representative workloads, measurements and reliability/security checks that should be included.
52. An operating system is being ported to a new processor family while retaining support for existing applications. Discuss the roles of abstraction, hardware-dependent code and interoperability.
53. Critically assess the statement: 'The best operating-system architecture is the one with the lowest overhead.' Use at least three design goals or system constraints in your argument.